

Program 1 :

Docker file : Multi-stage builds using builder and production stages.

```
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$ ls
Dockerfile  node_modules  package.json  package-lock.json  src
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$ cat Dockerfile
#Base Image Name
FROM node:20-alpine AS builder

#Label names of the project like who is the maintainer project name and project version
LABEL maintainer="abhi@gmail.com"
LABEL project="Demo project"
LABEL version="1.0"

#The project working directory
WORKDIR /app

#copying all the package.json files so that can be used to install all the packages required to run the project
COPY package*.json ./

#npm ci-- meaning clean install
RUN npm ci

#Copy the rest of the files of the project to the working directory
COPY . .

RUN npm run build

# 2nd build state --- named as production
FROM node:20-alpine AS production

#Setting up the working directory
WORKDIR /app

#copying all the required files from the first build stage
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

# Arguments that have been set so that can be imported in env variables..
ARG NODE_ENV=production

#Environmental variables that required to run the project
ENV NODE_ENV=$NODE_ENV
ENV PORT=4000

#expose the port of the docker container
EXPOSE 4000

#in the terminal run the command node server.js to start the express application
CMD ["node","dist/server.js"]

# #Health check route monitoring every 30s so check whether the app is running and working fine
HEALTHCHECK --interval=30s --timeout=10s \
    CMD curl -f http://localhost:4000/health || exit 1
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$
```

Docker build logs

Command: docker build -t program1 .

```
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$ docker build -t program2 .
[+] Building 2.3s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.47kB
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 44.92kB
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aee2af620ffa39f0a770265435
=> CACHED [builder 2/6] WORKDIR /app
=> CACHED [builder 3/6] COPY package*.json ./
=> CACHED [builder 4/6] RUN npm ci
=> CACHED [builder 5/6] COPY . .
=> CACHED [builder 6/6] RUN npm run build
=> CACHED [production 3/5] COPY --from=builder /app/package.json ./
=> CACHED [production 4/5] COPY --from=builder /app/dist ./dist
=> CACHED [production 5/5] COPY --from=builder /app/node_modules ./node_modules
=> exporting to image
=> => exporting layers
=> => writing image sha256:40e8906e4002daedd742a58ffeb07471a97f3cd8e7387e090336d375ec3d15ca
=> => naming to docker.io/library/program2
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$ docker run -d -p 4000:4000 program2
a1a14fbb7d08cdba7c48ec71fc8efbdb62695644cc5ec953cc9181508f78b03d
abhi@abhi-ThinkPad-T490:~/Desktop/DevopsLab/program2$
```

Running the docker built image in a container and mapping the port

command : docker run -d -p 3000:3000 program1

